MyLXC: A Docker-Inspired Wrapper for User-Friendly LXC Container Management

Nicholas P. Sweet

INTRODUCTION:

Base LXC is a powerful and lightweight containerization tool developed for Linux systems, but it requires users to manually configure and manage containers through numerous complex command line arguments. While LXC excels as a low-level containerizer, its bare bones interface creates unnecessary complexity and time sink for the user. This is the problem that I set out to solve when developing MyLXC. I wanted to create a Docker inspired wrapper that abstracts LXC's complex command line interface into simple intuitive commands, reducing repetition and making container management more accessible. Within the scope of this project, MyLXC was designed to implement five core commands: run to create and launch a container, start to resume a stopped container, stop to halt a running container, rm to remove a container, and ps to display active containers.

DESIGN:

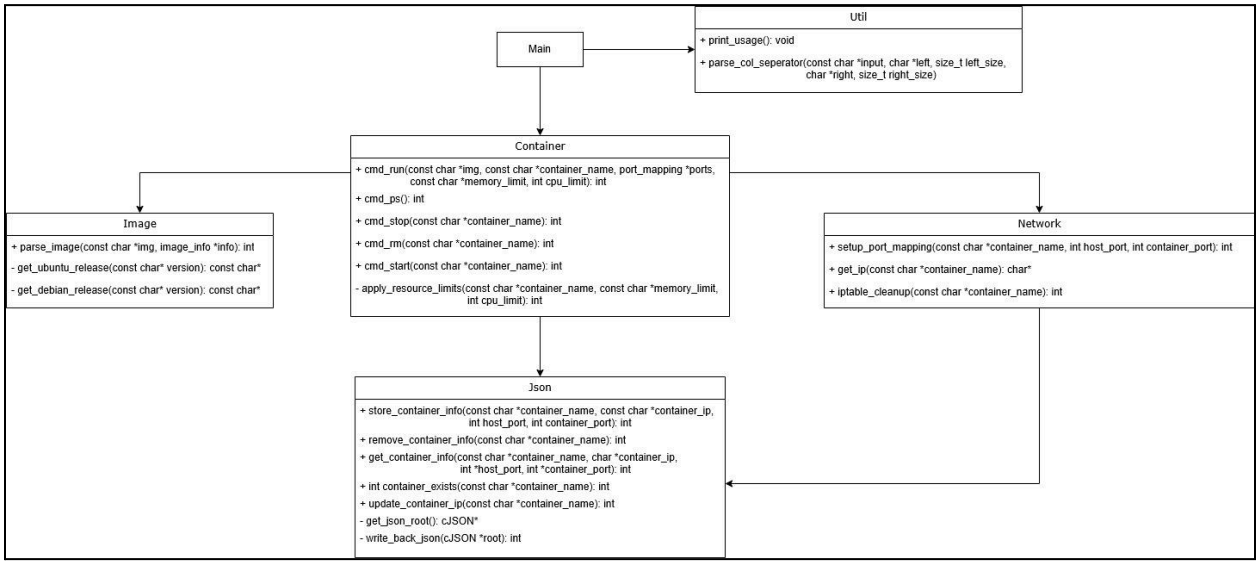
The "run" command takes an image in the format <distributor:version> which specifies the linux distributor and version. It is used later to get the release that is needed when creating an LXC container. The command supports four optional flags:

- "--name" - assigns a custom name to the container being created.
- "-p" - enables port forwarding in the format <host port:container port>, allowing network traffic to be redirected between the host and container.
- "--memory" - limits the container's memory usage, taking a value followed by M to denote megabytes (e.g. 512M).
- "--cpu" - limits the number of CPU cores available to the container, taking an integer value representing the core limit.

The commands "start", "stop", and "rm" commands each take one or more container names as arguments, allowing the user to start, stop, and remove multiple containers in a single command. The "ps" command takes no arguments and displays all active containers.

MyLXC is composed of six classes in addition to main, each responsible for a specific aspect of the system's functionality. Main handles command line arguments parsing and routes commands to appropriate handler functions in container class. The Container class is the core of MyLXC, responsible for executing the appropriate LXC commands based on the user's input. It processes the parsed command and its associated values, runs the corresponding LXC system commands, configures port mapping, and handles cleanup operations. The Image class is responsible for parsing the image that was provided, extracting the distributor and version to determine the release needed when creating an LXC container. The Network class handles configuring the necessary iptable rules to enable port forwarding between the host and the container. It also handles the cleanup of the iptable when the container is stopped. The Json class is used to add, edit, and remove port mappings to a JSON file, enabling easy iptable cleanup. Both Network and Container use the Json class to get the port mapping information for a container. Lastly, the Util class is where old versions of code and useful functions from previous iterations are stored, as well as the MyLXC usage instructions.

Class Diagram, illustrates the class structure and relationships between components:



IMPLEMENTATION:

MyLXC was implemented in C, leveraging the cJSON library for JSON parsing and manipulation within the Json class, and direct system calls to interact with LXC and iptables at the OS level.

Main, Image, Json, and Util do not interact with the OS through system calls, as they are responsible for argument parsing, image string processing, JSON manipulation, and utility functions. The Container class uses the LXC commands "lxc-create", "lxc-start", "lxc-stop", "lxc-destroy", and "lxc-ls --fancy" to implement the functions needed to handle the complete lifecycle of a container. The Network class, when port forwarding flags are provided, configures and runs the necessary iptable DNAT, FORWARD, and MASQUERADE rules needed to set up and clean up port forwarding.

The main challenges faced during the implementation of MyLXC were getting iptable rules to configure correctly and finding a reliable way to clean them up. When it came to configuring the iptable, the main issue was identifying the correct flags needed to set up port forwarding. Initially, a base configuration was quickly looked up without fully understanding the rules being used, which required spending a couple of days relearning the fundamentals before the iptable rules were configured correctly.

When it came to cleaning up the iptable, the original approach was to ask the user to provide the container's IP address rather than letting the container randomly generate it. The reason for this was to make cleanup and management easier. However, this added unnecessary complexity when interacting with MyLXC due to the need to add an ip flag to each command if a port forwarding is applied to the container. So randomly generating the IP and rather than performing a lookup every time cleanup was needed, JSON was used to persist the data needed to set up and remove port mappings. This worked well but occasionally produced duplicate iptable rules or duplicate entries for a container. The issue was traced back to the Container class calling the Network class multiple times, which was resolved by moving the call to a more appropriate location.

EVALUATION:

“**start” command:**

```
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo ./mylxc start  b c mylxc-3408
Starting container b...
Container b is running
Starting container c...
Container c is running
Starting container mylxc-3408...
Container mylxc-3408 is running
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$
```

Here the containers b, c, and mylxc-3408 were started using the one command.

“run” command:

```
}nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo ./mylxc run ubuntu:22.04 --name webserver -p 8080:80 --memory 512M --cpu 2
The cached copy has expired, re-downloading...
Downloading the image index
Downloading the rootfs
Downloading the metadata
The image cache is now ready
Unpacking the rootfs

---
You just created an Ubuntu jammy amd64 (20260429_07:42) container.

To enable SSH, run: apt install openssh-server
No default root or user password are set by LXC.
Starting container webserver...
Container webserver is running
Retrieving container webserver's IP address...
Container IP: 10.0.3.176
Setting up port forwarding: 8080 -> 10.0.3.176:80
Port mapping configured
webserver has had its memory limited to 512M
webserver has been limited to 2 CPUs
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ cat metadata.json
```

Here a container called webservers is created and port forwarded. It has been limited to 512 Megabytes of memory and 2 cores.

```
}nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:8080 to:10.0.3.176:80

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 38 packets, 7839 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  0    --  *      *       10.0.3.176           0.0.0.0/0
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$
```

This screenshot shows the iptable rules that were added once the run command finished.

“stop” command:

```
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo ./mylxc stop webserver
[sudo] password for nicholas-sweet:
Stopping container webserver...
Container stopped
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 1 packets, 281 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 593 packets, 290K bytes)
 pkts bytes target     prot opt in     out     source               destination
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$
```

Here the container webserver was stopped successfully and the associated iptable rules were cleaned up, confirming that port mapping cleanup works as intended.

“rm” command:

```
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo ./mylxc rm b c mylxc-3408
Removing container b...
Container was removed
Removing container c...
Container was removed
Removing container mylxc-3408...
Container was removed
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$
```

Here the containers b, c, and mylxc-3408 were removed using a single command, demonstrating MyLXC's ability to remove multiple containers at once.

“ps” command:

```
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$ sudo ./mylxc ps
[sudo] password for nicholas-sweet:
NAME       STATE   AUTOSTART GROUPS IPV4       IPV6 UNPRIVILEGED
a          RUNNING 0         -      10.0.3.64  -    false
b          RUNNING 0         -      10.0.3.177 -    false
c          RUNNING 0         -      10.0.3.143 -    false
mylxc-3408 RUNNING 0         -      10.0.3.114 -    false
nicholas-sweet@nicholas-sweet-VirtualBox:~/Documents/mylxc-Semi-Final$
```

Here all currently running containers are displayed, showing the output of the "ps" command at the time it was run.

DISCUSSION AND CONCLUSION:

MyLXC solves the issue of using numerous complex command line arguments to configure and manage containers. However, it only solves that issue for creating, starting, stopping, removing, and displaying containers. There are a lot more LXC commands that would need to be implemented to have full control over containers created. Future work would be implementing more of those commands to allow users to have more control over the containers. Overall, MyLXC achieves its core design goals and serves as a practical foundation for a more fully featured LXC management tool.